SCIF20002: Programming and Data analysis for Scientists Assessment 2

Mini Project and Report

The deadline for this assessment is **Wednesday 29th April 2026** at 12:30 p.m.

Further information related to this assessment and the methods used will be covered in the workshops in TB2B.

S. Hanna

Aims of the assessment:

For this assessment, you will be applying what you have learned in TB2 to a scientific problem involving epidemiology (the study of the spread of diseases). The aims are:

- To write programs in C++ or Python using object-oriented programming where appropriate, and use them to explore the problem;
- To demonstrate the use of version control through git;
- To provide a written report documenting how you have approached the problem, how you have tested your code, what calculations you have carried out and what you have found.

Notes:

- For this exercise, you will be required to submit one or more programs or Jupyter notebooks, and a report describing your results.
- You will be exploring solutions to the SEIR model (see below), which is a standard model of disease spread, using two basic techniques:
  - Direct solution of the ordinary differential equations involved;
  - Stochastic (Monte Carlo) simulations of the same system.
- Your code should be developed using version control (git) – we will be looking at your commit history as evidence of the development process of your code.
- You are free to use C++ or Python, or a mixture of both.
- It is important that you demonstrate use of object-oriented programming in your Monte Carlo simulations.

The SEIR model for disease transmission

Introduction

The SEIR model is known as a compartment model. The idea is that any given population is divided between a set of compartments. At any given time all members of the population will be in one of the compartments. In the present case the compartments are labelled:

S: Susceptible – member is able to catch the disease i.e. they are not immune;

E: Exposed – the member has been in contact with an infected individual, but is not yet displaying symptoms. This is the disease incubation period;

I: Infected – the member is infected and infectious;

R: Recovered – the member has recovered and is now immune from the disease.

The additional component of the model will be a set of probabilities that govern the likelihood of a particular member of the population transitioning from one compartment to the next. In the SEIR model, we assume that there is a one-way movement between the compartments, with the rates labelled as follows:

$$S \xrightarrow{\lambda} E \xrightarrow{\sigma} I \xrightarrow{\gamma} R \tag{1}$$

λ: Exposure rate (per day) – depends on number infected, I and the infection rate, β, in a population of N:

$$\lambda = \beta \frac{I}{N} \tag{2}$$

σ: Incubation rate (per day) – the rate at which exposed individuals begin to show symptoms;

γ: Recovery rate (per day) – the rate at which individuals move from the infectious category to the "R" category, which includes both those who recover and those that pass away as the result of the disease.

The SEIR equations

The SEIR equations are summarised as:

$$\frac{dS}{dt} = -\frac{\beta I}{N} S \tag{3}$$

The susceptible number goes down as individuals become exposed.

$$\frac{dE}{dt} = \frac{\beta I}{N} S - \sigma E \tag{4}$$

The exposed number increases due to susceptible individuals being exposed, but decreases as they move on to the infectious compartment.

$$\frac{dI}{dt} = \sigma E - \gamma I \tag{5}$$

The infected number increases as individuals move in from the exposed compartment, but decreases as they move on to the recovered compartment.

$$\frac{dR}{dt} = \gamma I \tag{6}$$

The "recovered" number increases as individuals move from the infectious compartment.

It may be more convenient to think in terms of reduced variables, where we eliminate the total population size, N:

$$s = \frac{S}{N}; \qquad e = \frac{E}{N}; \qquad i = \frac{I}{N}; \qquad r = \frac{R}{N}$$

The SEIR equations then become:

$$\frac{ds}{dt} = -\beta i s \tag{7}$$

$$\frac{de}{dt} = \beta i s - \sigma e \tag{8}$$

$$\frac{di}{dt} = \sigma e - \gamma i \tag{9}$$

$$\frac{dr}{dt} = \gamma i \tag{10}$$

During the Covid epidemic, much credence was given to the epidemic growth rate or "R" factor. In the SEIR model, this is defined as:

$$R_0 = \frac{\beta}{\gamma} s_0. \tag{11}$$

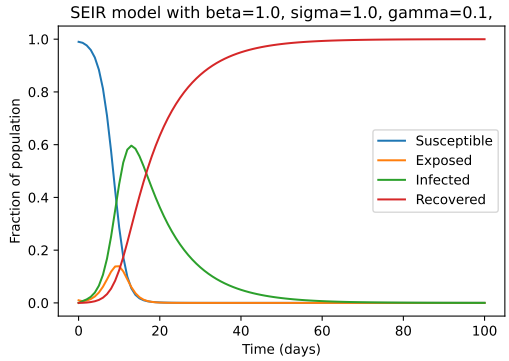
At the start of an epidemic, $s_0 \approx 1$ and $e_0 \approx 0$ so we can approximate the R-factor as $R_0 = \beta/\gamma$. Generally, when $R_0 < 1$ we expect any infections to quickly decay to zero. On the other hand, when $R_0 > 1$ we anticipate the outbreak will rise to some peak before finally decaying when most of the population have been infected.

Part 1: Solving the SEIR model as an initial value problem

In this part you will:

- Solve the coupled set of equations (7)–(10);
- Plot graphs of each of the compartment populations versus time for different choices of initial values and different values for the transition rates.

The SEIR model is an example of an initial value problem, similar to those we saw in the workshops. You have everything you need in equations (7)–(10) to set up a solution for the coupled set of equations. Write a program, in C++ or Python, to solve the SEIR equations. Verify that it is working by comparing your results with the figure shown here, using $\beta = 1$, $\sigma = 1$ and $\gamma = 0.1$ *per day* as a first guess at suitable model parameters, and $s_0 = 0.99$, $e_0 = 0.01$, $i_0 = 0.0$ and $r_0 = 0.0$ as initial values. These values correspond to $1\,\%$ of the population being infected at the outset.



Once you are confident that your code is working, explore the behaviour of the model for different initial conditions and different combinations of β, σ and γ. In particular, you should explore parameter combinations for which the initial growth rate, R_0, implies outbreaks will persist, or decay. Discuss your findings in your report.

Part 2: A Brownian analogue of the SEIR model

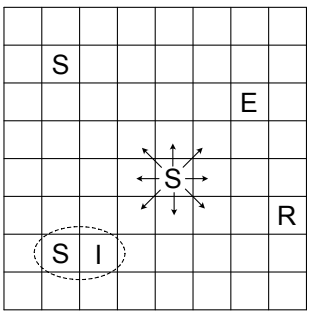
In this part of the assessment, you will:

- Develop an analogue of the SEIR model using a Monte Carlo simulation of the Brownian motion of infectious agents.
- Develop an object oriented approach to modelling the agents, with one attribute of each agent being the compartment (susceptible, exposed, infected, recovered) that it currently belongs in.
- Compare your results with the stochastic approach with the initial value solution in **Part 1**.

In the workshops, we looked at a stochastic (Monte Carlo) model for a chain reaction, in which neutrons moved across a lattice at random, interacting with the different lattice sites depending on what they contained. The aim of the present exercise is to use a very similar approach to model a population of agents spreading a disease. A lattice is used as a way of limiting the possible arrangements of the infectious agents and simplifying the job of finding infected neighbours. Each agent moves across the lattice and stores its current infectious state (compartment) on the lattice. Checking for an infected neighbour is then as simple as looking at the contents of the adjacent sites on the lattice. Further details of the model follow.

Infectious agents

Use object oriented programming to create the infectious agents. Each agent will need to store (i) its lattice coordinates and (ii) its current compartment. Each time it moves the agent will need to update its coordinates, and update the lattice as well. The agent should also be able to check neighbouring lattice sites for infectious agents and update its SEIR compartment when needed. The figure on the right shows a square lattice with a small number of agents. Agents should be able to move to adjacent lattice sites, provided they are empty, or else remain unmoved. In the lower left of the lattice a susceptible agent, S, is next to an infectious agent, I, and so is expected to become exposed, E.

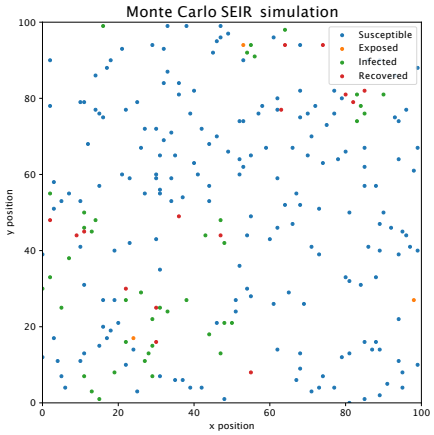


Lattice sites are either empty, or they contain agents that are in one of the SEIR compartments. It is simple to represent each state by a number e.g. 0 = empty, 1 = susceptible, 2 = exposed, 3 = infected and 4 = recovered. As the agents move across the lattice, the infection process is simulated by checking for infected neighbours. Each time an agent moves next to an infected neighbour, it is at risk of being exposed. We recall from Eq. (2) that the exposure rate λ depends on both the infection rate β and the number of infected agents. In the Monte Carlo model, the exposure rate will be determined by the number of collisions with infected agents, which will depend on the density of agents on the lattice. It is sufficient to set $\beta = 1$ – lower values will simply reduce the probability of infection spread.

Once an agent has become exposed, it will randomly progress from exposed to infected, and from infected to recovered, depending on the chosen values for σ and γ.

Initialisation of the model

Create the 2D lattice, and populate it with agents. The agents should be placed randomly on the lattice. The larger fraction of them should be in the susceptible state, with a small number of exposed individuals. In the figure shown here, 250 agents were placed on a 100 $\times$ 100 lattice, with P(susceptible)=0.95 and P(exposed) = 0.05. The snapshot is taken after about 100 Monte Carlo steps (MCS).

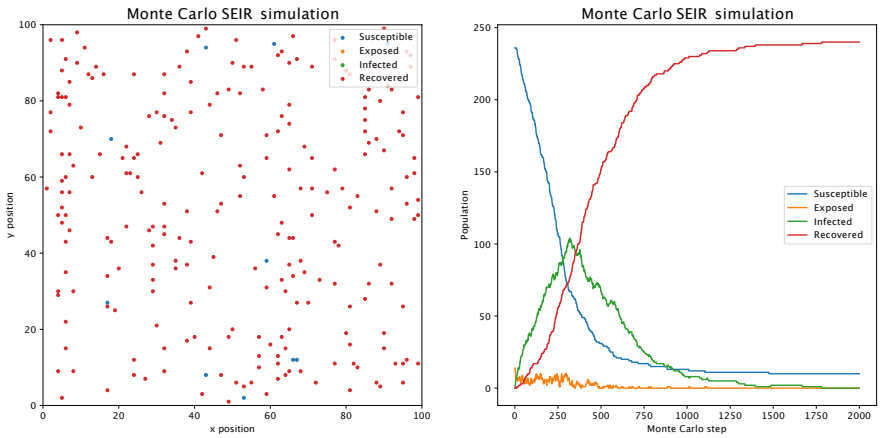


The Monte Carlo step

For each MCS you should proceed as follows:

1. For each agent in turn you should do the following in turn:
   (a) Attempt to move the agent in a random direction to a neighbouring cell of the lattice. If the site is already occupied, the agent should stay in its original position. If the move is successful, remember to update the lattice. If an agent attempts to move off the edge of the lattice, either use a hard wall to prevent it, or periodic (wraparound) boundary conditions to allow it.
   (b) If the agent is either exposed or infected, generate a random number to decide whether it should move on to the next compartment, i.e. either infected or recovered, respectively. You will need to compare your random number with the chosen values of σ and γ. Note that these values will be different from those used in **Part 1** because they have different units (*per day* in Part 1 and *per MCS* in the current part).
   (c) If the agent is susceptible, examine its nearest neighbours to determine whether it should become infected. If using periodic boundaries, remember your checks will need to wraparound as well.
2. Once the agents are all updated, count up the number in each compartment and store these numbers in arrays for plotting.

The plot below shows results after 2000 MCS for a simulation with the parameters discussed above, with $\sigma = 0.1$ per MCS and $\gamma = 0.005$ per MCS. The random fluctuations visible in the plots are in contrast with the smooth curves observed in Part 1.



The Investigation

As with the model in **Part 1**, you should explore this model by varying the parameters and find conditions for which an outbreak occurs and for which it dies away. You have slightly different parameters to work with here, because the number density of agents on the lattice will play a role in the initial exposure which previously depended only on β. See if you can model the same situations using both approaches.

Finally, consider situations that the SEIR equations do not cover. For example, in Covid, it is clear that a small fraction of recovered individuals could be re-infected. How would you handle this in your Monte Carlo model?

Your submission

You must write up your investigation as a report which documents:

1. How your code works. This is in addition to writing good clear comments in your code. Your Monte Carlo code must use object-oriented programming. You must justify any choices you have made regarding the Monte Carlo procedure. There should be instructions on how to compile and run your code, as well as documentation of expected output.

2. How you have tested your code. This is a complex problem, so you need to make sure your code behaves as expected for a simplified test case. Comparison with the known behaviour of the SEIR model is useful here.

3. The results of your investigations. This must be an analysis of what you have found, and not just a dump of data!

Your code should be developed using version control, and submitted as a **git bundle** along with your report. Only code (and associated README files) should be submitted in this bundle! **Your report should be submitted as a separate document.** Your submissions should use only your student number for the file name.

The deadline for submission is Wednesday, 29th April 2026 (Week 24) at 12:30pm.

Mark scheme

As this is the terminal assessment for the unit, marks and feedback will not be released until after the exam period, in line with University procedure.

The assessment will be marked according to the following criteria.

Competence	Basic	Intermediate	Advanced
Numerical methods	* A mostly functional code has been produced, but there are flaws in the implementation. * A rudimentary attempt to test the code has been attempted, but it is neither rigorous nor complete. * An attempt to carry out the required investigation has been made. It is either incomplete, or lacks the evidence to support the conclusions.	* A functional Monte Carlo code has been produced. Choices of paramater are arbitrary, incorrect or not justified. * The code has been reasonably tested, but the evidence to support this is either unclear or lacking. * A successful investigation has been carried out. The range of cases studied is limited, and/or the evidence is to support the conclusions is unclear	* A fully-functional piece of code has been produced, with a well-justified set of parameters. * The code is demonstrated to work correctly for a sensible choice of test case, and clear but concise evidence is presented. * A detailed study has been undertaken, with a clear and concise presentation of the findings. Appropriate use of tools to analyse and interpret the data has been made.
Software engineering	* The code base contains a git repository with one branch	* The repository has at least five commits with meaningful messages demonstrating a logical progression * Appropriate usage of `.gitignore`	* The code uses feature branches (at least two separate branches) and merging * The code base uses advanced features (e.g. Makefiles, wrapping /binding with Python)
Object-Oriented Programming	* There exists at least one basic class	* The code is organised in multiple (at least two) objects that interact with each other * The code base is organised in separate definition and implementation files	* The classes use well-motivated and different levels of encapsulation (private and public) * The final user interacts primarily with the custom classes (e.g. most functions called by the main() function are member functions of some custom object)
Documentation	* The code is sparsely commented, but enough to guide the user. The report gives some insight into the choices made for the code. * The report is acceptably written, with an attempt made at all the required tasks. Elements of the report are unclear or hard to follow.	* The code is commented to a good standard, but elements of its functionality are unclear. * The investigation has been carried out and documented adequately. Evidence is presented to support the required tasks. The report is mostly easy to follow, but small sections may be unclear.	* The code is clearly and concisely document, making it clear to the user how it is designed and used. * The report is clearly written and easy to understand. A good balance of written explanation and interpretation of data is present along with appropriate figures.

A submission that only meets all of the basic criteria will be awarded a mark of 40. A mark of 70+ will only be awarded for submissions demonstrating all of the advanced criteria.